UNIVERSIDAD POLITÉCNICA BICENTENARIO

Práctica 03. Medición de temperatura con DS18B20 y LCD 16×2 I2C

Presenta:

Jesús Ángel Bonilla Duran

Viridiana Santibáñez Carrillo

Jocelyn Rodriguez Torres

Matricula:

24033407

24033066

24033436

Asignatura:

Sistemas Embebidos

Profesor:

Hugo Antonio Méndez Guzmán

Silao 15 de octubre de 2026

Contenido

Introducción 2
Marco Teórico 3
Sistemas programables 3
Microcontrolador ESP32 3
Simulación en Wokwi 4
Control de temperatura mediante histéresis 4
Desarrollo 4
Material requerido 4
Funcionamiento general del programa 6
Descripción del código 6
Resultados 11
Implementación del sistema en físico 11
Observaciones y Conclusiones 13
Referencias 14

Lista de Figuras

Figura 1 Simulación del sistema de control de temperatura mediante ESP32 en Wokwi, utilizando LEDs para representar los actuadores de enfriamiento y calentamiento. 6
Figura 2 Conexión del sensor DS18B20 con el ESP32.. 11
Figura 3 Implementación del sistema de calentamiento y enfriamiento 12
Figura 4 y Figura 5. Conexiones de los botones pulsadores para configurar los límites de temperatura. 12
Figura 6 Conexión de la pantalla LCD para visualizar la temperatura. 13
Figura 7 y Figura 8 Evidencia complementaria del montaje de los botones pulsadores para configurar los límites de temperatura. 13

Introducción

La regulación de temperatura es una aplicación importante de los sistemas embebidos, ya que permite mantener una variable física dentro de un intervalo establecido mediante la lectura de sensores y el control de actuadores este tipo de sistemas puede utilizarse en hornos, incubadoras, cámaras térmicas y procesos de calentamiento o enfriamiento.

En esta práctica se desarrolló un sistema de control de temperatura con una tarjeta ESP32, un sensor digital DS18B20 y una pantalla LCD de 16 × 2 caracteres con interfaz I²C. El sistema permite visualizar la temperatura medida y modificar los límites inferior y superior mediante tres botones pulsadores, de acuerdo con la lectura del sensor, el programa activa el sistema de calentamiento o el de enfriamiento.

Para implementar el control se utilizó Visual Studio Code con PlatformIO y Arduino Framework también se emplearon las bibliotecas OneWire y DallasTemperature para establecer la comunicación con el sensor, así como LiquidCrystal_I2C para controlar la pantalla la programación incorpora una lógica de histéresis que cambia entre calentamiento y enfriamiento al alcanzar los límites configurados.

El objetivo de la práctica fue desarrollar un sistema embebido capaz de medir, visualizar y regular la temperatura mediante entradas digitales, procesamiento de información y control de actuadores. con ello, se integraron conocimientos de programación de microcontroladores, comunicación entre dispositivos electrónicos y automatización de procesos..

Marco Teórico

Sistemas programables

Un sistema programable es un dispositivo electrónico que ejecuta instrucciones almacenadas en un programa para realizar una función determinada. Su comportamiento puede modificarse mediante cambios en el código, lo que permite adaptarlo a distintas condiciones de funcionamiento sin necesidad de rediseñar todo el circuito.

En esta práctica, el ESP32 recibe la información del sensor DS18B20, procesa la temperatura medida y determina el estado de los actuadores. También interpreta las pulsaciones de los botones para modificar los límites de control y actualiza la información presentada en la pantalla LCD.

Microcontrolador ESP32

El ESP32 es un microcontrolador desarrollado por Espressif Systems que dispone de entradas y salidas digitales, temporizadores y diferentes interfaces de comunicación para conectar sensores y dispositivos externos.

En el sistema desarrollado, el ESP32 funciona como unidad central de control. Mediante el GPIO 4 recibe los datos del sensor de temperatura; los GPIO 21 y 22 se utilizan para la comunicación I2C con la pantalla LCD; los GPIO 25, 26 y 27 reciben las señales de los botones, y los GPIO 19 y 23 controlan los módulos de relé (Nick, s.f.).

Simulación en Wokwi

Wokwi es una plataforma de simulación electrónica que permite programar microcontroladores y representar circuitos con componentes compatibles. Su utilización facilita la revisión de conexiones y de la lógica del programa antes de realizar el montaje físico.

En esta práctica se utilizaron LEDs para representar visualmente las señales de los actuadores de calentamiento y enfriamiento. Esta representación permitió observar los cambios de las salidas del ESP32 según la temperatura simulada. Sin embargo, los LEDs no reproducen la transferencia de calor del foco ni el comportamiento térmico de la celda Peltier, por lo que estas características deben evaluarse mediante pruebas físicas (Group, 2023).

Control de temperatura mediante histéresis

La histéresis es una estrategia de control que emplea dos límites de referencia para determinar cuándo cambia el estado de un sistema. En lugar de utilizar un único valor para activar y desactivar una salida, se establecen un límite inferior y uno superior que definen una banda de operación.

En el programa se establecen los siguientes parámetros iniciales:

- **LL:** límite inferior de temperatura, establecido en 20 °C.
- **HL:** límite superior de temperatura, establecido en 35 °C.

Cuando la temperatura alcanza o supera HL, se activa el estado de enfriamiento. Cuando desciende hasta LL o un valor inferior, el programa cambia al estado de calentamiento. Entre ambos límites, se conserva el estado anterior del control. Por ello, en la implementación realizada, la banda de histéresis no implica que ambos actuadores permanezcan apagados (PlatformIO, s.f.).

Desarrollo

Material requerido

- 1 tarjeta ESP32.
- 1 sensor digital de temperatura DS18B20.
- 1 pantalla LCD de 16 × 2 caracteres con interfaz I2C.
- 3 botones pulsadores.
- 1 módulo de relé para el sistema de calentamiento.

- 1 módulo de relé para el sistema de enfriamiento.
- 1 foco o elemento calefactor compatible con el montaje.
- 1 celda Peltier y ventiladores para el enfriamiento.
- Protoboard, si se utilizó en el montaje.
- Cables de conexión.
- Computadora con Visual Studio Code y PlatformIO.
- Bibliotecas OneWire, DallasTemperature y LiquidCrystal_I2C.
- Simulador Wokwi, si se realizaron pruebas virtuales.

Tabla 1. Configuración de pines

Elemento	Pin GPIO	Configuración
Sensor DS18B20	GPIO 4	Entrada de temperatura mediante OneWire
Relé de calentamiento	GPIO 19	Salida digital
Relé de enfriamiento	GPIO 23	Salida digital
Botón SUBIR	GPIO 25	Entrada INPUT_PULLUP
Botón BAJAR	GPIO 26	Entrada INPUT_PULLUP
Botón SELECT	GPIO 27	Entrada INPUT_PULLUP
Pantalla LCD	GPIO 21 y 22	Comunicación I2C (SDA y SCL)

Posteriormente, se realizó la simulación del circuito en Wokwi para verificar el funcionamiento del sistema de control de temperatura, debido a que el simulador no cuenta con todos los componentes requeridos para representar el montaje físico, se utilizaron cuatro LEDs azules para simular la celda Peltier y los tres motores del sistema de enfriamiento, así como un LED rojo para representar el foco del sistema de calentamiento.

Estos elementos permiten visualizar la activación de las salidas controladas por el ESP32 mediante los módulos de relé, de acuerdo con los límites de temperatura establecidos en el programa.

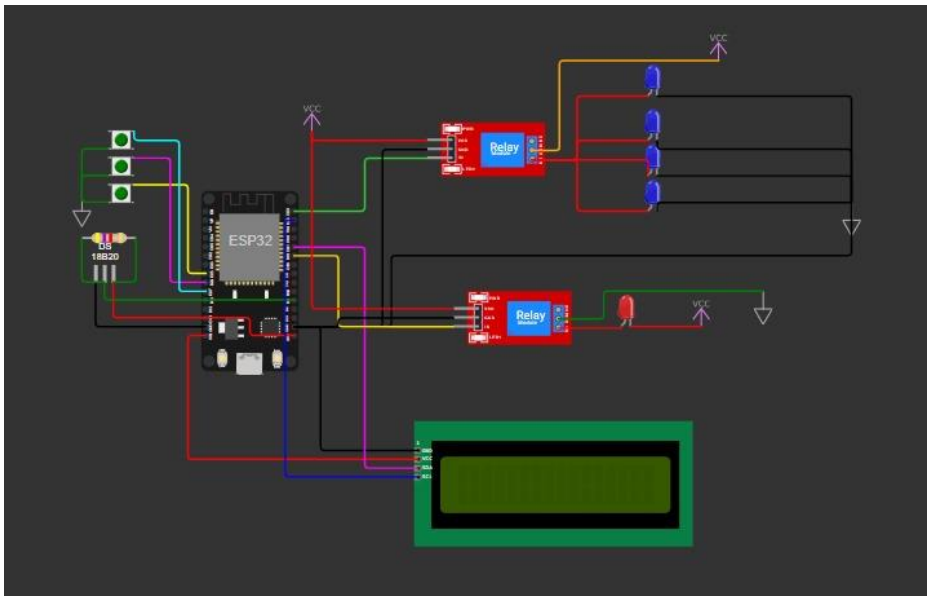


Figura 1 Simulación del sistema de control de temperatura mediante ESP32 en Wokwi, utilizando LEDs para representar los actuadores de enfriamiento y calentamiento.

Funcionamiento general del programa

El programa se organizó en cuatro etapas principales: inicialización de los dispositivos, lectura de los botones, adquisición de la temperatura y control de los actuadores.

Al iniciar, el ESP32 configura los pines de los relés como salidas y escribe un nivel HIGH para mantenerlos apagados, suponiendo que los módulos de relé utilizados se activan con nivel LOW los botones se configuran como entradas con resistencias pull-up internas; además, se inicializan el sensor y la pantalla LCD, los límites de temperatura comienzan en 20 °C para LL y 35 °C para HL.

Durante la ejecución, los botones permiten seleccionar y modificar los límites el sensor proporciona las mediciones de temperatura, que se comparan con los valores establecidos finalmente, el programa actualiza la pantalla LCD para mostrar los límites, la temperatura medida y el estado del sistema.

Descripción del código

La siguiente tabla explica las instrucciones y variables principales utilizadas en el archivo main.cpp.

Instrucción o elemento	Función

`#include <Arduino.h>`	Incorpora las funciones básicas del Arduino Framework.
`#include <Wire.h>`	Permite utilizar la comunicación I²C.
`#include <LiquidCrystal_I²C.h>`	Facilita el control de la pantalla LCD mediante I²C.
`#include <OneWire.h>`	Incorpora las funciones de comunicación OneWire.
`#include <DallasTemperature.h>`	Facilita la lectura del sensor DS18B20.
`#define PIN_DS18B20 4`	Asigna el GPIO 4 a la línea de datos del sensor.
`#define FOCO_CALENTAR 19`	Asigna el GPIO 19 al control del relé de calentamiento.
`#define ENFRIAMIENTO 23`	Asigna el GPIO 23 al control del relé de enfriamiento.
`#define BOTON_SUBIR 25`	Define el pin del botón para aumentar los valores.
`#define BOTON_BAJAR 26`	Define el pin del botón para disminuir los valores.
`#define BOTON_SELECT 27`	Define el botón para alternar entre LL y HL.
`int LL = 20;`	Establece el límite inferior inicial en 20 °C.
`int HL = 35;`	Establece el límite superior inicial en 35 °C.
`LiquidCrystal_I²C lcd(0x27, 16, 2);`	Configura la pantalla con dirección I²C 0x27, 16 columnas y 2 filas.
`OneWire oneWire(PIN_DS18B20);`	Crea el objeto de comunicación OneWire asociado al GPIO 4.
`DallasTemperature sensors(&oneWire);`	Configura el objeto que administra las lecturas del DS18B20.
`bool enfriando = false;`	Guarda el estado del control; inicialmente corresponde al modo de calentamiento.
`bool seleccionLL = true;`	Indica que el límite inferior está seleccionado inicialmente.
`pinMode()`	Configura cada pin como entrada o salida.
`digitalWrite()`	Establece el nivel lógico de las salidas que controlan los relés. En este programa se usa HIGH para apagarlos y LOW para activarlos, suponiendo que los módulos son activos en nivel bajo.
`digitalRead()`	Lee el estado lógico de los botones pulsadores.
`INPUT_PULLUP`	Activa la resistencia pull-up interna de una entrada digital.
`sensors.begin()`	Inicializa la biblioteca del sensor de temperatura.
`sensors.setResolution(11)`	Configura la resolución del sensor a 11 bits.
`sensors.setWaitForConversion(false)`	Permite solicitar la conversión sin bloquear la ejecución; se debe verificar que la lectura se tome después de que la conversión haya terminado.
`sensors.requestTemperatures()`	Solicita una nueva conversión de temperatura.
`sensors.getTempCByIndex(0)`	Obtiene la temperatura en grados Celsius del primer sensor detectado.
`DEVICE_DISCONNECTED_C`	Permite identificar una lectura que indica desconexión del sensor.
`millis()`	Devuelve el tiempo transcurrido desde el inicio del programa en milisegundos y permite programar tareas periódicas sin usar pausas bloqueantes.
`ultimoTiempoBoton`	Permite limitar la frecuencia con la que se procesan las pulsaciones.
`estadoAnteriorSubir`, `estadoAnteriorBajar` y `estadoAnteriorSelect`	Guardan los estados previos de los botones para detectar una nueva pulsación.
`seleccionLL = !seleccionLL;`	Cambia entre la selección del límite inferior y el superior.
`LL++, LL--, HL++ y HL--`	Modifican los límites en incrementos de un grado, respetando las condiciones programadas.

tiempoParpadeo	Controla el intervalo de parpadeo del límite seleccionado.
mostrarSeleccion	Determina si el límite seleccionado se muestra o se oculta temporalmente.
lcd.init()	Inicializa la pantalla LCD.
lcd.backlight()	Enciende la iluminación de fondo de la pantalla.
lcd.clear()	Borra el contenido de la pantalla.
lcd.setCursor()	Establece la posición donde se mostrará el siguiente texto.
sprintf()	Da formato al texto que contiene los límites de temperatura.
dtostrf()	Convierte la temperatura de tipo flotante a una cadena de caracteres para mostrarla.
lcd.print()	Presenta los valores y mensajes en la pantalla.

A continuación, se muestra el código empleado para el desarrollo de la práctica.

main.cpp

```
#include <Arduino.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <OneWire.h>
#include <DallasTemperature.h>
#define PIN_DS18B20 4
#define FOCO_CALENTAR 19
#define ENFRIAMIENTO  23
#define BOTON_SUBIR  25
#define BOTON_BAJAR  26
#define BOTON_SELECT 27
int LL = 20;
int HL = 35;
LiquidCrystal_I2C lcd(0x27, 16, 2);
OneWire oneWire(PIN_DS18B20);
DallasTemperature sensors(&oneWire);
bool enfriando = false;
bool seleccionLL = true;

bool estadoAnteriorSubir  = HIGH;
bool estadoAnteriorBajar  = HIGH;
bool estadoAnteriorSelect = HIGH;

unsigned long tiempoParpadeo    = 0;
unsigned long tiempoLectura     = 0;
unsigned long tiempoLCD         = 0;
unsigned long ultimoTiempoBoton = 0;

bool mostrarSeleccion = true;
float temperatura = 25.0;
void setup() {
  pinMode(FOCO_CALENTAR, OUTPUT);
  pinMode(ENFRIAMIENTO, OUTPUT);

  digitalWrite(FOCO_CALENTAR, HIGH);
```

```
  digitalWrite(ENFRIAMIENTO, HIGH);
  pinMode(BOTON_SUBIR, INPUT_PULLUP);
  pinMode(BOTON_BAJAR, INPUT_PULLUP);
  pinMode(BOTON_SELECT, INPUT_PULLUP);
  sensors.begin();
  sensors.setResolution(11);
  sensors.setWaitForConversion(false);
  sensors.requestTemperatures();
  Wire.begin(21, 22);
  lcd.init();
  lcd.backlight();
  lcd.clear();
}
void loop() {
  bool estadoSubir  = digitalRead(BOTON_SUBIR);
  bool estadoBajar  = digitalRead(BOTON_BAJAR);
  bool estadoSelect = digitalRead(BOTON_SELECT);

  if (millis() - ultimoTiempoBoton >= 120) {
        if (estadoSelect == LOW && estadoAnteriorSelect == HIGH) {
      seleccionLL = !seleccionLL;
      mostrarSeleccion = true;
      tiempoParpadeo = millis();
      ultimoTiempoBoton = millis();
    }
    else if (estadoSubir == LOW && estadoAnteriorSubir == HIGH) {
      if (seleccionLL) {
        if (LL < HL - 1) LL++;
      } else {
        if (HL < 100) HL++;
      }
      ultimoTiempoBoton = millis();
    }
    else if (estadoBajar == LOW && estadoAnteriorBajar == HIGH) {
      if (seleccionLL) {
        if (LL > 0) LL--;
      } else {
        if (HL > LL + 1) HL--;
      }
      ultimoTiempoBoton = millis();
    }
  }

  estadoAnteriorSubir  = estadoSubir;
  estadoAnteriorBajar  = estadoBajar;
  estadoAnteriorSelect = estadoSelect;
  if (millis() - tiempoParpadeo >= 400) {
    tiempoParpadeo = millis();
    mostrarSeleccion = !mostrarSeleccion;
  }

  if (millis() - tiempoLectura >= 375) {
    tiempoLectura = millis();
    float tempLeida = sensors.getTempCByIndex(0);
```

```
    if (tempLeida != DEVICE_DISCONNECTED_C && tempLeida > -50.0) {
      temperatura = tempLeida;
    }
    sensors.requestTemperatures();
    if (temperatura >= HL) {
      enfriando = true;
    } else if (temperatura <= LL) {
      enfriando = false;
    }
    if (enfriando) {
      digitalWrite(FOCO_CALENTAR, HIGH);
      digitalWrite(ENFRIAMIENTO, LOW);
    } else {
      digitalWrite(ENFRIAMIENTO, HIGH);
      digitalWrite(FOCO_CALENTAR, LOW);      }
  }
  if (millis() - tiempoLCD >= 150) {
    tiempoLCD = millis();

    char buffer[10];
    lcd.setCursor(0, 0);
    if (seleccionLL && !mostrarSeleccion) {
      lcd.print("LL=--C ");
    } else {
      sprintf(buffer, "LL=%-2dC ", LL);
      lcd.print(buffer);
    }
    lcd.setCursor(8, 0);
    dtostrf(temperatura, 4, 1, buffer);    lcd.print("T=");
    lcd.print(buffer);
    lcd.print("C ");
    lcd.setCursor(0, 1);
    if (!seleccionLL && !mostrarSeleccion) {
      lcd.print("HL=--C ");
    } else {
      sprintf(buffer, "HL=%-2dC ", HL);
      lcd.print(buffer);
    }

    lcd.setCursor(8, 1);
    if (enfriando) {
      lcd.print("[ENFR.] ");
    } else {
      lcd.print("[CALEN] ");
    }
  }
}
```

Resultados

Implementación del sistema en físico

Después de realizar la simulación en Wokwi, se llevó a cabo el montaje físico del sistema de control de temperatura. Se conectaron el ESP32, el sensor DS18B20, la pantalla LCD con comunicación I²C, los botones de ajuste y los elementos encargados del calentamiento y el enfriamiento.

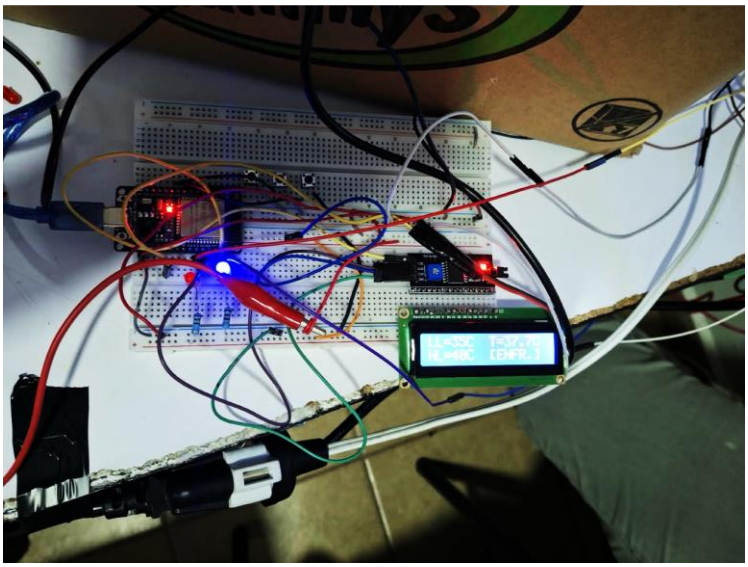


Figura 2 Conexión del sensor DS18B20 con el ESP32..

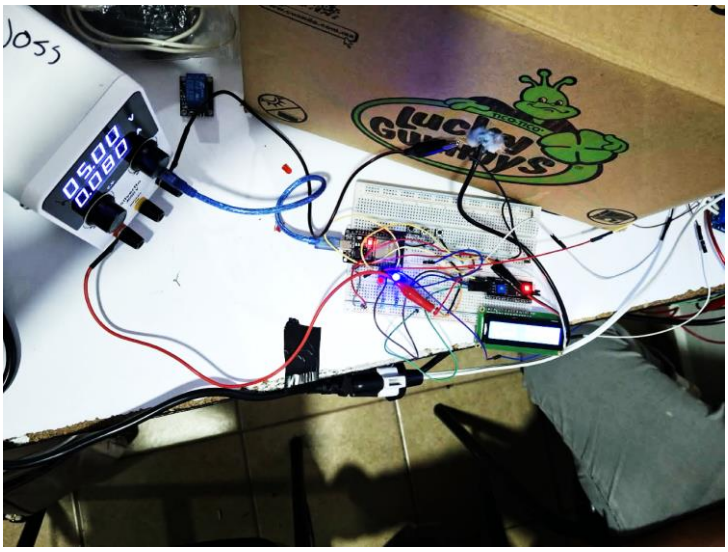
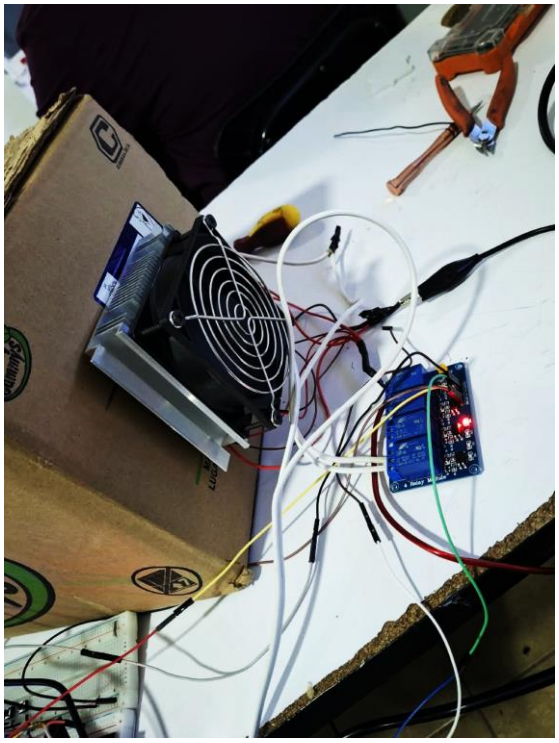


Figura 3 Implementación del sistema de calentamiento y enfriamiento



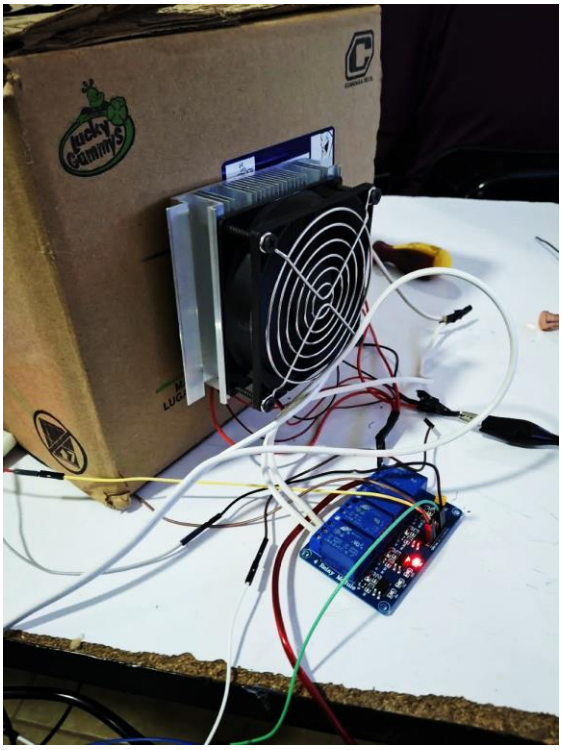


Figura 4 y Figura 5. Conexiones de los botones pulsadores para configurar los límites de temperatura.

Durante las pruebas, el programa permitió establecer la lógica de funcionamiento del sistema de control de temperatura con el ESP32 y el sensor DS18B20. Los tres botones pulsadores permiten configurar los límites inferior y superior, y la pantalla LCD muestra los valores y la temperatura medida.

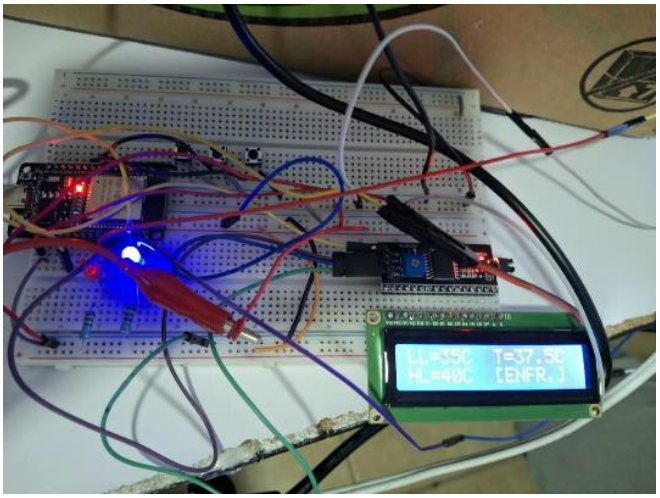


Figura 6 Conexión de la pantalla LCD para visualizar la temperatura.

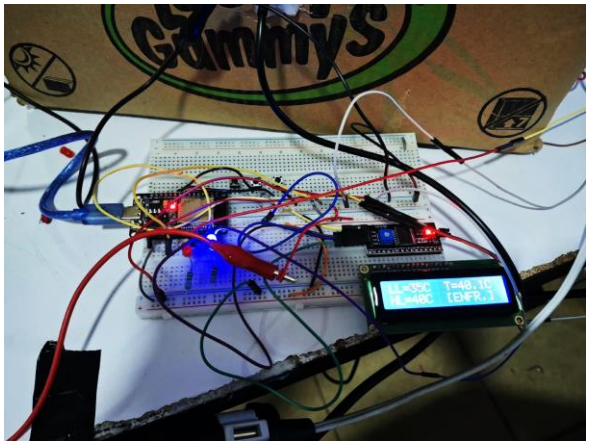




Figura 7 y Figura 8 Evidencia complementaria del montaje de los botones pulsadores para configurar los límites de temperatura.

Observaciones y Conclusiones

Durante la práctica se comprobó la importancia de configurar correctamente los pines del ESP32 y las bibliotecas necesarias para conectar el sensor DS18B20 y la pantalla LCD, también se observó que las entradas configuradas con INPUT_PULLUP presentan un nivel lógico LOW cuando se presionan los pulsadores.

La programación de los límites permitió modificar las condiciones de operación sin cambiar las conexiones físicas, además, se estableció una diferencia mínima de un grado entre LL y HL para evitar que ambos límites fueran iguales o se cruzaran.

En cuanto al control térmico, la lógica de histéresis conserva el estado anterior mientras la temperatura permanece entre los límites, sin embargo, el código mantiene activo el calentamiento cuando el estado de enfriamiento es falso por ello, el sistema no cuenta con una condición en la que ambos actuadores estén apagados dentro de esa banda; esta decisión debe valorarse según la aplicación.

También es importante considerar el tiempo de conversión del DS18B20 el programa solicita nuevas conversiones sin esperar de forma bloqueante y consulta la temperatura cada 375 ms, dado que el sensor se configuró a 11 bits, conviene comprobar que cada lectura corresponda a una conversión terminada y que las mediciones sean consistentes.

En esta práctica se desarrolló un sistema embebido para medir y controlar la temperatura mediante un ESP32, un sensor digital DS18B20 y una pantalla LCD de 16 × 2 caracteres con interfaz I²C, la integración de estos componentes permitió relacionar la medición de una variable física con el procesamiento de datos y la generación de señales para controlar actuadores.

Los tres botones pulsadores permiten modificar los límites inferior y superior durante la operación, la pantalla LCD facilita la consulta de la temperatura medida, los límites configurados y el estado del sistema los módulos de relé funcionan como interfaz de control entre las salidas del ESP32 y los elementos de calentamiento y enfriamiento.

El desarrollo permitió aplicar conocimientos de programación de microcontroladores, comunicación digital y control automático, también mostró la importancia de coordinar las lecturas del sensor y definir con claridad las condiciones de conmutación de los actuadores.

Como mejora, se recomienda sincronizar las solicitudes y lecturas del sensor para confirmar que la conversión haya terminado antes de utilizar el dato, también conviene incorporar una respuesta segura ante lecturas inválidas y revisar si la aplicación requiere una zona de operación en la que ambos actuadores permanezcan apagados, estas modificaciones ayudarían a mejorar la confiabilidad del prototipo.

Referencias

[1] (Nick, s.f.) *The ESP32 chip explained: Advantages and applications*.

[2] PlatformIO. (s.f.). PlatformIO: Professional collaborative platform for embedded development.

[3] Maxi. (s.f.). Introducción a PlatformIO con Visual Studio Code.

[4] Group, u. (2023). Wokwi: Simulador en línea de Arduino, ESP32, STM32 y Raspberry Pi Pico.